

LING 510: Week 5 review

10/02/2026

Shaunak Phadnis

sphadnis@umass.edu

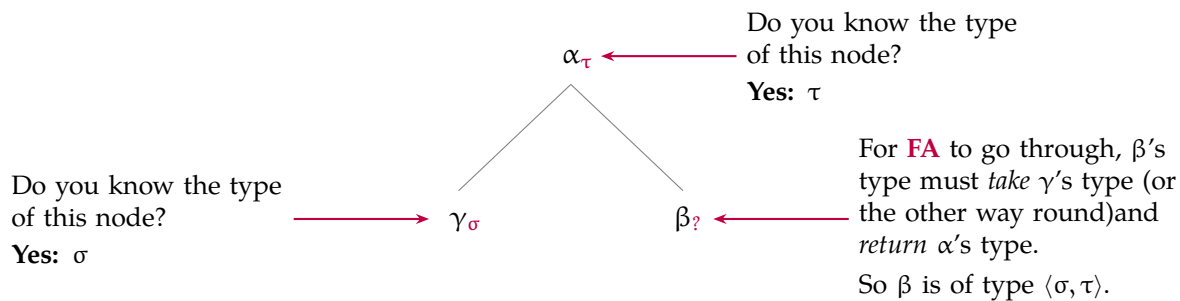
Office hours: W 11–1PM, N421H | [Zoom](#)



Where are we?

- Principle of compositionality and Syntax
- Type driven system of interpretation that assigns denotations to syntactic objects.
- Via rules of interpretation (Lexicon, TN, NN, FA)
- Turns out composition is just function application.

We have an inventory of types. Given a tree, we can guess what type a particular constituent should have based on its syntactic positions and the rule of function application. Here's a simple way to do it.



Remember that in a tree with multiple nodes with unknown types, you always know that a sentence is type t ., names are type e , that intransitives are type $\langle e, t \rangle$. For all other nodes, you can infer their types recursively by the method described above.

Apart from that we also observed that

Natural language conjunction^a is *flexible*. It can conjoin different kinds of constituents (as long as they are the same type!).

- Is there a separate entry for and_{NP} , an and_{VP} , an and_S , an and_{AdvP} , etc. except that all of them are spoken the same way as 'and'?
- If so, then every time we discover new types of constituents that can be conjoined, we would need to add a new entry for conjunction corresponding to that type in the lexicon.
- But all of these conjunction do the same thing: they take two constituents of the same type and return a constituent of that same type!

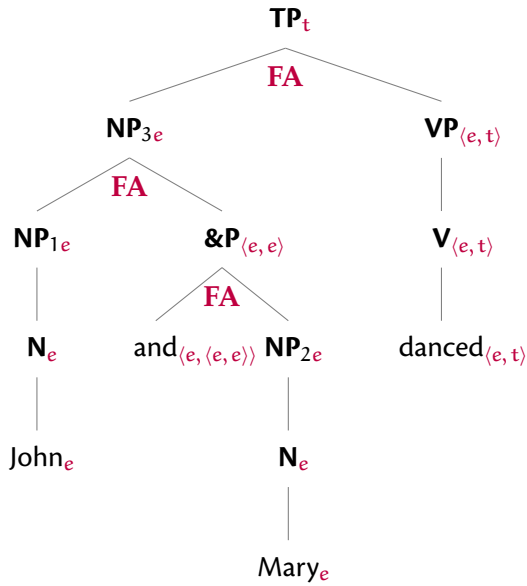
Is that good design? Could we do better?

^aAlso disjunction

Here's a demonstration.

(1) John and Mary danced.

NP-conjunction



Lexical entries

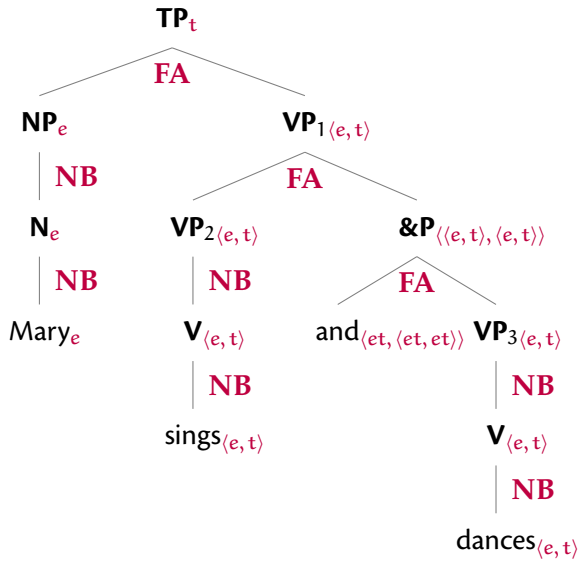
$\llbracket \text{John} \rrbracket = j$
 $\llbracket \text{Mary} \rrbracket = m$
 $\llbracket \text{and} \rrbracket = [\lambda x : x \in D_e. [\lambda y : y \in D_e. y \text{ and } x]]$
 $\llbracket \text{danced} \rrbracket = [\lambda x : x \in D_e. T \text{ iff } x \text{ danced}]$

Composition

$\llbracket \&P \rrbracket = [\lambda y : y \in D_e. y \text{ and } m]$ **FA**
 $\llbracket NP_3 \rrbracket = j \text{ and } m$ **FA**
 $\llbracket VP \rrbracket = [\lambda x : x \in D_e. T \text{ iff } x \text{ danced}]$ **NB**
 $\llbracket TP \rrbracket = T \text{ iff } j \text{ and } m \text{ danced}$ **FA**

(2) Mary sings and dances.

VP-conjunction



Lexical entries

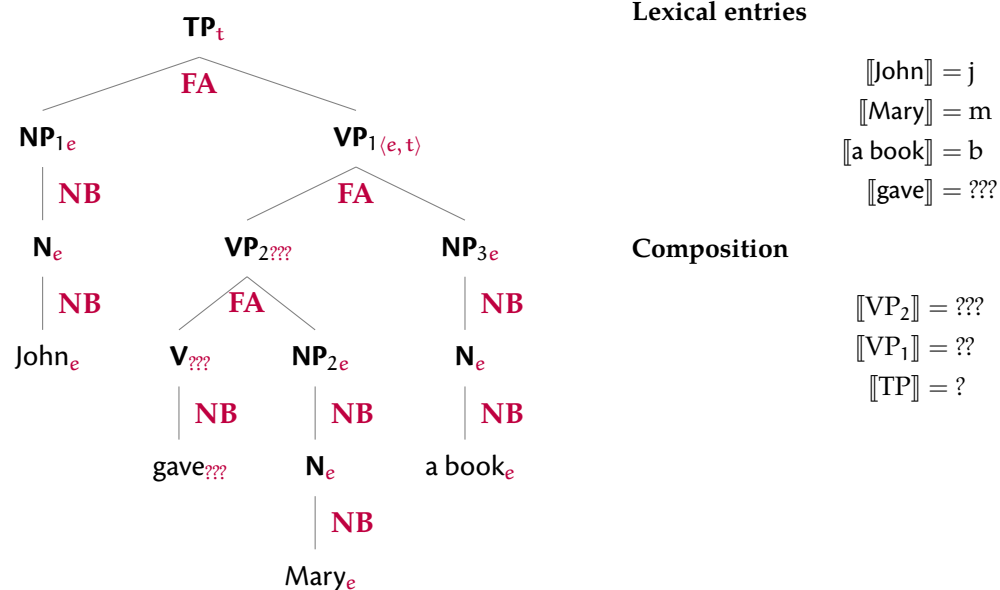
$\llbracket \text{Mary} \rrbracket = m$
 $\llbracket \text{sings} \rrbracket = [\lambda x : x \in D_e. T \text{ iff } x \text{ sings}]$
 $\llbracket \text{dances} \rrbracket = [\lambda x : x \in D_e. T \text{ iff } x \text{ dances}]$
 $\llbracket \text{and} \rrbracket = [\lambda f : f \in D_{\langle e, t \rangle}. [\lambda g : g \in D_{\langle e, t \rangle}. [\lambda x : x \in D_e. T \text{ iff } g(x) = T \text{ and } f(x) = T]]]$

Composition

$\llbracket \&P \rrbracket = [\lambda g : g \in D_{\langle e, t \rangle}. [\lambda x : x \in D_e. T \text{ iff } g(x) = T \text{ and } x \text{ dances}]]$ **FA**
 $\llbracket VP_1 \rrbracket = [\lambda x : x \in D_e. T \text{ iff } x \text{ sings and } x \text{ dances}]$ **FA**
 $\llbracket TP \rrbracket = T \text{ iff } m \text{ sings and } m \text{ dances}$ **FA**

Last week we also figured out the type for the ditransitive verb *give*. Tell me the type again and figure out the right lexical entry that yields the correct truth conditions.

(3) John gave Mary a book.



- John gave Mary a book is true just in case John was the agent of giving, a book was the thing being given, and Mary was the recipient.
- In order to write the lexical entry for gave, we need to capture this three-place relation between the agent (John), the beneficiary (Mary), and the theme (a book).
- But note that the internal argument of gave *does not* merge first in this structure.
- So when we write the lexical entry, the order of variables introduced should respect this order of composition.

Which of the following is the correct entry for gave? what would the output be for each?

1. [[gave]] = $[\lambda x_e. [\lambda y_e. [\lambda z_e. \text{T iff } x \text{ gave } y \text{ to } z]]]$
2. [[gave]] = $[\lambda x_e. [\lambda y_e. [\lambda z_e. \text{T iff } z \text{ gave } y \text{ to } x]]]$
3. [[gave]] = $[\lambda x_e. [\lambda y_e. [\lambda z_e. \text{T iff } y \text{ gave } x \text{ to } z]]]$

Subproof a. VP₂

1. $\left[\left[\begin{array}{c} \text{VP}_{2\langle e, \langle e, t \rangle \rangle} \\ \swarrow \quad \searrow \\ \text{V}_{\langle e, \langle e, \langle e, t \rangle \rangle} \quad \text{NP}_{2e} \\ | \quad \quad | \\ \text{NB} \quad \quad \text{NB} \\ \text{gave}_{\langle e, \langle e, \langle e, t \rangle \rangle} \quad \text{Mary}_e \end{array} \right] \right] = \left[\left[\begin{array}{c} \text{V}_{\langle e, \langle e, \langle e, t \rangle \rangle} \\ | \\ \text{NB} \\ \text{gave}_{\langle e, \langle e, \langle e, t \rangle \rangle} \end{array} \right] \left(\left[\begin{array}{c} \text{NP}_{2e} \\ | \\ \text{NB} \\ \text{Mary}_e \end{array} \right] \right) \right] \quad (\text{by FA})$
2. = $\llbracket \text{gave} \rrbracket (\llbracket \text{Mary} \rrbracket)$ (by NB)
3. = $[\lambda x_e. [\lambda y_e. [\lambda z_e. \top \text{ iff } z \text{ gave } y \text{ to } x]]](m)$ (by lexical entries)
4. = $[\lambda y_e. [\lambda z_e. \top \text{ iff } z \text{ gave } y \text{ to } m]]$ (by β -reduction)

Subproof b. VP₁

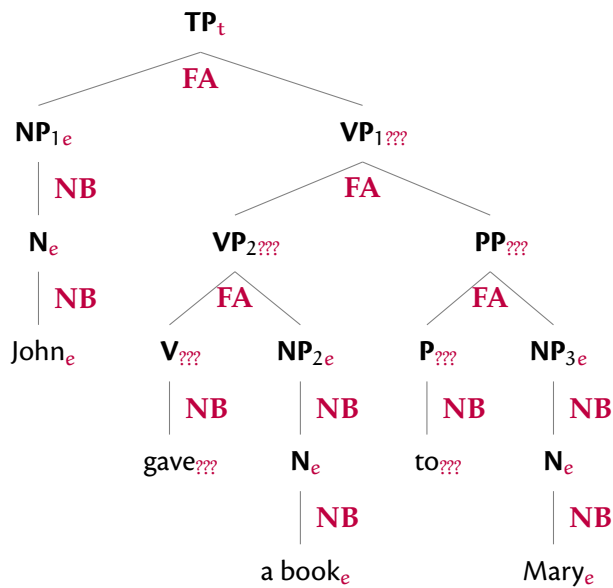
1. $\left[\left[\begin{array}{c} \text{VP}_{1\langle e, t \rangle} \\ \swarrow \quad \searrow \\ \text{VP}_{2\langle e, \langle e, t \rangle \rangle} \quad \text{NP}_{3e} \\ \swarrow \quad \searrow \quad \swarrow \quad \searrow \\ \text{gave} \quad \text{Mary} \quad \text{a book} \end{array} \right] \right] = \left[\left[\begin{array}{c} \text{VP}_{2\langle e, \langle e, t \rangle \rangle} \\ \swarrow \quad \searrow \\ \text{gave} \quad \text{Mary} \end{array} \right] \left(\left[\begin{array}{c} \text{NP}_{3e} \\ \swarrow \quad \searrow \\ \text{a book} \end{array} \right] \right) \right] \quad (\text{by FA})$
2. = $[\lambda y_e. [\lambda z_e. \top \text{ iff } z \text{ gave } y \text{ to } m]](b)$ (by a., NB)
3. = $[\lambda z_e. \top \text{ iff } z \text{ gave } b \text{ to } m]$ (by β -reduction)

Derivation. TP

1. $\left[\left[\begin{array}{c} \text{TP}_t \\ \swarrow \quad \searrow \\ \text{NP}_{1e} \quad \text{VP}_{1\langle e, t \rangle} \\ \swarrow \quad \searrow \quad \swarrow \quad \searrow \\ \text{John} \quad \text{gave} \quad \text{Mary} \quad \text{a book} \end{array} \right] \right] = \left[\left[\begin{array}{c} \text{VP}_{1\langle e, t \rangle} \\ \swarrow \quad \searrow \\ \text{gave} \quad \text{Mary} \quad \text{a book} \end{array} \right] \left(\left[\begin{array}{c} \text{NP}_{1e} \\ \swarrow \quad \searrow \\ \text{John} \end{array} \right] \right) \right] \quad (\text{by FA})$
2. = $[\lambda z_e. \top \text{ iff } z \text{ gave } b \text{ to } m](j)$ (by b., NB)
3. = $\top \text{ iff } j \text{ gave } b \text{ to } m$ (by β -reduction)

So John gave Mary a book is true iff John gave a book to Mary. Di-transitives like *give* also exhibit what is called ‘dative alternation’. These are also called ‘double object constructions’ (see [Larson 1988](#) for instance) given in (4)

- (4) a. John gave Mary a book.
- b. John gave a book to Mary.



Your task

Work *top down*. Fill in the types of higher VPs then,

- to Mary is a PP. What type must the PP be for **FA** to apply at the higher VP? _____
- Now what type is to? _____
- What type is the lower VP? _____
- What type is gave? _____

Given

[[John]] = j
[[Mary]] = m
[[a book]] = b

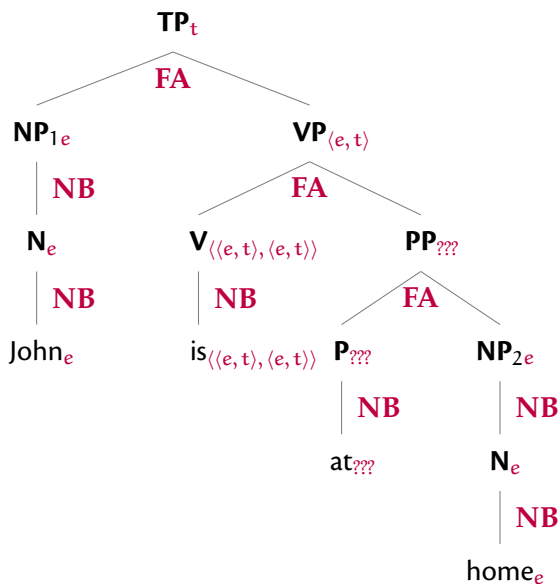
Write down

[[gave]] = _____
[[to]] = _____

The structure now has a PP hosting the indirect object. Would it affect our computation? What role is *to* playing here? How does it affect its argument?

Now consider the preposition in (5). If we supposed that the entry and type for *at* is the same as *to* in (4), what will happen? What do we predict its role to be in the sentence?

(5) John is at home.



Given

[[John]] = j
[[home]] = h
[[is]] = $[\lambda f : f \in D_{(e,t)} \cdot f]$

Your task

[[at]] = ??

- is is an identity function on predicates. Given its type, what type must PP be? _____
- What type is at? _____
- Derive [[TP]], labelling each step. What are the truth conditions for (5)?

References

Larson, Richard K. 1988. On the double object construction. *Linguistic Inquiry* 19(3). 335–391. <http://www.jstor.org/stable/25164901>.